# Homework 2

Embedding Transfer Learning, Retrieval-Augmented Generation, and Training Optimization

| Student | Configuration |
|---|---|
| Kavan Siddesh Kumar | SID4 9486 \| SEED 9486 \| SLICE 486 \| HP_ID 0 \| CLS_A 6 \| CLS_B 0 |

## 1. Personal parameters

SEED=9486 was used for stochastic operations, including NumPy and PyTorch seeding, Word2Vec `seed`, Transformers `set_seed`, and t-SNE `random_state`. SLICE=486 and HP_ID=0 are reported for standing personal-parameter requirements but were not used to alter HW2 experiments because HW2 does not define an HP_ID mapping. CLS_A=6 and CLS_B=0 are reported only.

## 2. Part 1 — Embedding-based transfer learning

### 2.1 Objective and data

The goal was to start from Google News `word2vec-google-news-300`, inspect neighbors for cast, score, plot, screen, and review, fine-tune a new Word2Vec model on IMDB review text, then measure neighbor change and each target word's own vector movement. The IMDB CSV has 50,000 reviews. In Colab it was read from `/content/IMDB Dataset.csv` with `engine="python"` after a default parser failure; configuration records `data/IMDB Dataset.csv`. The first 10,000 reviews were used. Tokenization lowercased text, replaced HTML break tags with spaces, removed non-alphanumeric characters except apostrophes, and split on whitespace.

| Setting | Observed value |
|---|---|
| Pretrained model | word2vec-google-news-300 (vocab 3,000,000; dim 300) |
| Reviews used | 10,000 |
| Window / min_count / epochs | 5 / 2 / 5 |
| Workers / seed | 4 / 9486 |
| Shared words initialized | 26,556 |

### 2.2 Baseline and post-fine-tuning neighbors

Baseline top-3 neighbors (before fine-tuning).

| Word | N1 | Sim | N2 | Sim | N3 | Sim |
|---|---|---|---|---|---|---|
| cast | casts | 0.7219 | casting | 0.7188 | Cast | 0.6638 |
| score | scoring | 0.7197 | scores | 0.6596 | scored | 0.6384 |
| plot | plots | 0.7625 | Plot | 0.6524 | plotting | 0.6328 |
| screen | screens | 0.7729 | onscreen | 0.6115 | LCD_screen | 0.5599 |
| review | reviewed | 0.6630 | reviewing | 0.6610 | reviews | 0.6380 |

Top-3 neighbors after IMDB fine-tuning.

| Word | N1 | Sim | N2 | Sim | N3 | Sim |
|---|---|---|---|---|---|---|
| cast | casting | 0.6610 | actors | 0.6380 | performances | 0.6271 |
| score | soundtrack | 0.6945 | cinematography | 0.6470 | music | 0.6255 |
| plot | storyline | 0.7121 | story | 0.6920 | script | 0.6227 |
| screen | screens | 0.5805 | stage | 0.4860 | lebowski | 0.4396 |
| review | comment | 0.6588 | reviews | 0.6376 | recommendation | 0.5854 |

## 2.3 Vector shift, figure, and interpretation

Shift is defined as `1 − cosine_similarity(original, fine-tuned)` for the same word. Lower cosine similarity means larger movement in embedding space.

| Word | Cosine similarity (orig. vs fine-tuned) | Shift |
|---|---|---|
| review | 0.609045 | 0.390955 |
| score | 0.622409 | 0.377591 |
| screen | 0.676173 | 0.323827 |
| plot | 0.701515 | 0.298485 |
| cast | 0.781051 | 0.218949 |

Most shifted: **review** (similarity 0.609045, shift 0.390955). Least shifted: **cast** (similarity 0.781051, shift 0.218949). Neighbors moved from morphological/general associates toward movie-review language (for example, score → soundtrack/cinematography/music; plot → storyline/story/script).

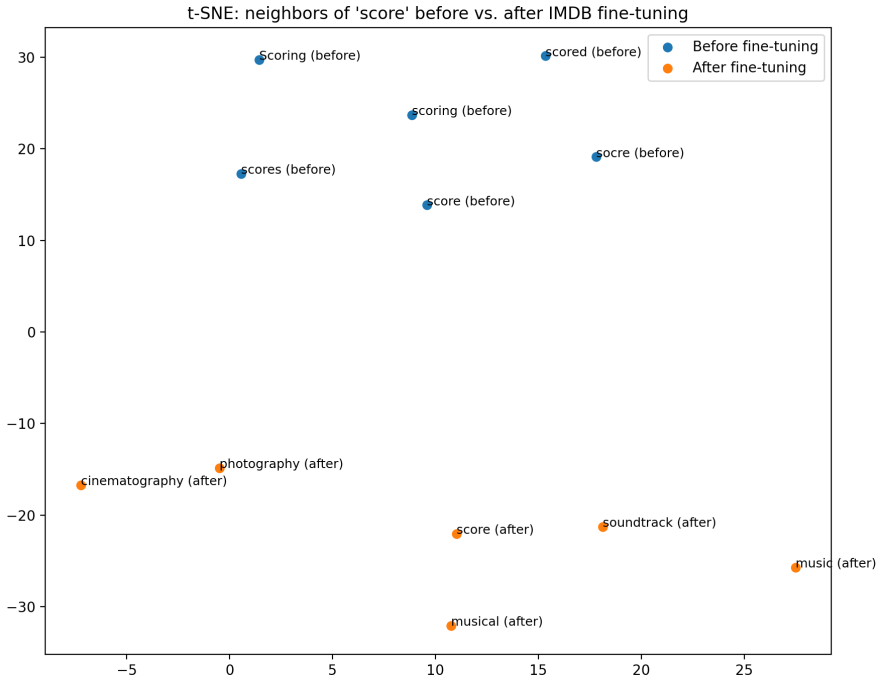


Figure 1. t-SNE of neighbors of “score” before versus after IMDB fine-tuning.

Limitations: multi-worker Word2Vec can still vary across environments; only 10,000 reviews were used; cosine shift measures a word’s own vector movement rather than neighbor-list quality; and t-SNE is a 2-D projection that can distort distances.

# 3. Part 2 — Retrieval-augmented generation

## 3.1 Sources, loader, and pipeline

Ten movie pages were required: Inception, The Godfather, Titanic (1997 film), The Dark Knight, Pulp Fiction, Forrest Gump, The Matrix, Interstellar, Parasite (2019 film), and Gladiator (2000 film). `WikipediaLoader` failed with a `JSONDecodeError` against the Wikipedia API in Colab. Documents were therefore loaded with LangChain `WebBaseLoader` using the official English Wikipedia URLs. Ten documents loaded successfully.

| Component | Observed setting |
|---|---|
| Primary chunk size / overlap | 500 / 50 (3,411 chunks) |
| Alternate chunk size / overlap | 800 / 100 (Inception & Godfather only) |
| Embedding model | sentence-transformers/all-MiniLM-L6-v2 |
| Vector store | FAISS |
| Retriever | FAISS as_retriever(k=3) |
| Prompt / context | Explicit PromptTemplate; concatenated page_content |
| LLM | google/flan-t5-base (text2text-generation) |

The RAG pipeline kept PromptTemplate, retriever, context formatting, and the LLM call separate (no RetrievalQA chain).

## 3.2 Questions, retrieved passages, and answers

**Q1.** Who directed Inception, and what is its central plot concept?
**Answer:** Christopher Nolan
Rank 1 (Inception / https://en.wikipedia.org/wiki/Inception): opening synopsis naming Christopher Nolan and describing a subconscious heist / idea-implantation plot. Ranks 2–3 were review commentary and reference lines. First relevant rank **1**. The answer named the director but omitted the plot concept present in rank 1.

**Q2.** Who played Don Vito Corleone in The Godfather?
**Answer:** Marlon Brando
Rank 1 casting passage states Marlon Brando was chosen to portray Vito Corleone. First relevant rank **1**.

**Q3.** What ship is central to the story of Titanic?
**Answer:** Titanic
All three retrieved passages were footnote/reference lines about the film rather than a clear encyclopedic statement identifying RMS Titanic. Automatic keyword matching for “rms titanic” failed, but manual inspection likewise finds the top-3 uninformative for the ship question, so this is treated as a retrieval failure rather than a keyword-only failure.

**Q4.** Who is the main antagonist in The Dark Knight?
**Answer:** Gambol
Top-3 passages were sequel/reference snippets mentioning Gambol, not the Joker. The model answered Gambol, which is incorrect for the main antagonist. Genuine RAG failure.

**Q5.** What major Oscar award did Parasite win?
**Answer:** Best Picture
Rank 2 states Parasite won Best Picture (among four Oscars). First relevant rank **2**.

## 3.3 Alternate chunking, success rate, and failures

| Question | 500/50 answer | 800/100 answer | Top-chunk change |
|---|---|---|---|
| Inception director/plot | Christopher Nolan | Christopher Nolan | Synopsis → DiCaprio production quote |
| Godfather / Vito Corleone | Marlon Brando | Marlon Brando | Similar casting passages |

| Question | First relevant rank | Success |
|---|---|---|
| Inception | 1 | Yes |
| The Godfather | 1 | Yes |
| Titanic | — | No |
| The Dark Knight | — | No |
| Parasite | 2 | Yes |

**Retrieval Success Rate: 3/5 = 60.00%** (matches artifacts/part2/rag_configuration.json).

**Failure 1 — The Dark Knight.** Retrieved chunks were citation fragments about Gambol rather than Joker-centered plot/cast text. With that context, flan-t5-base answered “Gambol.” Failure mode: noisy HTML/reference retrieval plus generation from misleading context.

**Failure 2 — Titanic.** Top-3 chunks were reference footnotes without a clear ship-identity passage. WebBaseLoader also ingested Wikipedia chrome, producing 3,411 chunks and diluting useful content. Failure mode: low-value reference retrieval despite a short answer (“Titanic”) that can be guessed from the page title. Additional limitation: Inception rank-1 context contained both director and plot idea, but the LLM returned only “Christopher Nolan.”

# 4. Part 3 — Training optimization

## 4.1 Shared setup

Executed on Google Colab with CUDA available, GPU Tesla T4, PyTorch 2.11.0+cu128, device cuda, and seed 9486. Shared synthetic classification setup: input dim 128, hidden dim 256, output dim 10, batch size / effective batch 64, 40 training steps, Adam, learning rate 0.001. Gradient accumulation used 4 × 16 micro-batches. Tensor-creation benchmark used shape (4096, 512) for 25 iterations.

## 4.2 Results for all five techniques

| Technique | Configuration | Device | Time (s) | Peak mem (MB) | Final loss |
|---|---|---|---|---|---|
| Tensor creation | CPU tensor creation | cpu | 0.323885 | — | — |
| Tensor creation | GPU tensor creation | cuda | 0.012795 | 1059.211 | — |
| Weight init | PyTorch default | cuda | 0.286958 | 1061.305 | 2.309902 |
| Weight init | Xavier uniform | cuda | 0.062525 | 1061.305 | 2.318031 |
| Checkpointing | No checkpointing | cuda | 0.134194 | 1070.089 | 2.303427 |
| Checkpointing | Activation checkpointing | cuda | 0.260050 | 1071.089 | 2.303427 |
| Grad. accum. | Standard training | cuda | 0.069160 | 1062.305 | 2.309902 |
| Grad. accum. | 4 × 16 micro-batches | cuda | 0.212517 | 1062.305 | 2.328997 |
| Mixed precision | Full precision | cuda | 0.066441 | 1062.305 | 2.309902 |
| Mixed precision | CUDA mixed precision | cuda | 0.319262 | 1062.306 | 2.310822 |

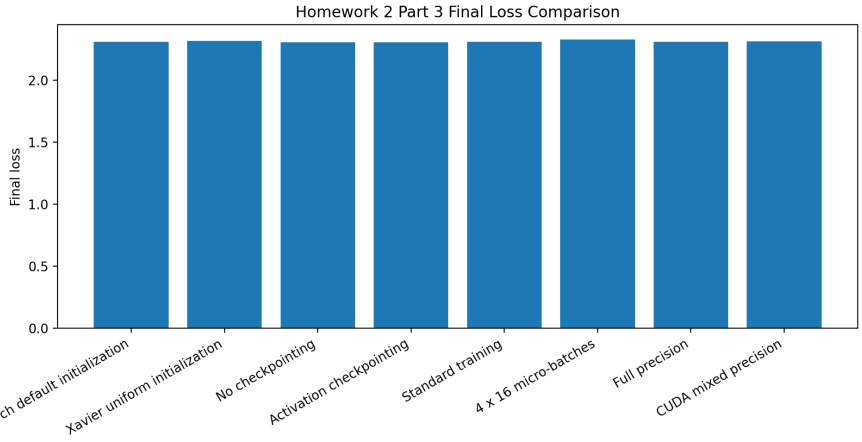


Figure 2. Final-loss comparison across Part 3 training configurations.

### 4.3 Findings and hardware notes

GPU tensor creation was much faster than CPU (0.0128 s vs 0.3239 s). Final losses across training comparisons stayed near about 2.30–2.33 on this short synthetic task. Xavier finished faster than default initialization in this run. Activation checkpointing increased wall time (0.260 s vs 0.134 s) without reducing measured peak allocated memory on this small DeepMLP. Gradient accumulation and mixed precision also increased time relative to their matched baselines; mixed-precision peak memory was essentially unchanged at this model scale.

CUDA and mixed-precision measurements were available on the Tesla T4 runtime; CPU peak memory for tensor creation was recorded as unavailable. These micro-benchmarks are sensitive to warm-up, allocator state, and model size, so absolute memory rankings should be interpreted cautiously. No HW2 checkpoints were written under checkpoints/.

## 5. AI use summary

AI assistance was used to organize the notebook/repository, simplify implementation code, draft documentation, set reproducibility cells, debug Colab errors, and format the report. I personally executed the notebook in Google Colab, verified the dataset, inspected retrieved passages and answers, identified RAG failures, reviewed numerical results against artifacts, and checked the final submission. Detailed debugging examples (IMDB ParserError, missing WINDOW, Transformers text2text-generation pin, malformed accelerate install text, WikipediaLoader JSONDecodeError → WebBaseLoader, missing RecursiveCharacterTextSplitter import) are recorded in AI_USE.md.